TEXT ANALYTICS

Evaluation Protocols

Francesco Spinnato

* francesco.spinnato@unipi.it
University of Pisa



Why Evaluation Matters

The goal of a text analytics model is not to perform well on the documents we already have, but to **generalize to documents we have never seen.**

To estimate how well a model generalizes, we must evaluate it on data that was not used for learning.

Two things have to be decided:

- **what** we measure → *metrics*
- **how** we measure it → *experimental protocols*

Losses and Metrics

What the model optimizes, and what we measure

Loss vs Metric

- A **loss function** is used during training to fit the model.
  - to optimize the parameters;
- An **evaluation metric** is used to measure performance after training.
  - and to choose the hyperparameters.

They answer different questions:

- Loss: *"What does the model optimize?"*
- Metric: *"How good is the model?"*

They are often related, but not necessarily the same. For example:

- Cross-entropy is differentiable and therefore trainable;
- F1 is what you actually care about, and is not.

Parameters and Hyperparameters

- A **parameter** is learned from the data during training, by minimizing the loss.
- A **hyperparameter** is set **before** training: it controls the structure of the model or how it is trained, and the training data cannot choose it.

Model	Parameters	Hyperparameters
Logistic regression	weights $\mathbf{w}$, bias b	regularization strength, class weights
Naive Bayes	$\hat{P}(c)$, $\hat{P}(w \mid c)$	the smoothing constant (the $+1$ of Laplace)
k-means	centroids $\boldsymbol{\mu}_j$	number of clusters K
LDA	topics, topic mixtures	number of topics k
TF-IDF vectorizer	vocabulary, IDF weights	n-gram range, minimum document frequency

Hyperparameters are chosen by comparing trained models on a **metric**, computed on data the models did not see.

The vectorizer has parameters too: it is *fit* on the data, so it falls under the same rules as the model.

The Losses We Have Met

Models trained so far minimized a loss, written here for a **single** example (ℓ).

Task	Model	Loss ℓ
Regression	linear regression	squared error: $(y - \hat{y})^2$
Binary classification	logistic regression	binary cross-entropy: $-\left[\, y \ln \hat{P} + (1-y) \ln(1-\hat{P}) \,\right]$
Multiclass classification	softmax regression	cross-entropy: $-\ln \hat{P}(y \mid \mathbf{x})$
Clustering	k-means	squared distance to its centroid: $\|\mathbf{x} - \boldsymbol{\mu}_j\|^2$
Language and topic models	n-grams, LDA	negative log-likelihood: $-\ln P(w_i \mid \text{context})$

Aggregating over the instances gives you the overall loss $\mathcal{L}$, e.g.,

- averaging the squared error $\rightarrow$ **mean squared error (MSE)**;
- summing the k-means term over all documents gives the within-cluster **sum of squares**, called **inertia**.

Maximizing the likelihood of the data is the same as minimizing the **negative log-likelihood**. Cross-entropy is a negative log-likelihood too: minus the log of the probability the model gave to the right answer.

When the Loss Is Also the Metric

Sometimes the loss *is* the metric. **Perplexity** is the exponentiated average negative log-likelihood per word:

$$PP(D) = \sqrt[T]{\frac{1}{P(w_1 \dots w_T)}} = \exp\Big(-\frac{1}{T} \sum_{i=1}^{T} \ln P(w_i \mid w_1 \dots w_{i-1}) \Big)$$

The quantity is the same; what changes is **where it is computed**:

- **training** text, it rewards memorization: a larger n-gram always lowers it
- **held-out** text, it measures generalization

Training loss always rewards a more *flexible* model. This is why evaluation needs **unseen data**, and often a **different measure**.

Metrics Depend on the Task

The choice of evaluation metric depends on:

- the **task** (regression, classification, clustering, topic modeling, ...)
- the **data** (e.g. class imbalance, which is the norm in text)
- the **application** (which kinds of errors are expensive)

Flagging a legitimate email as spam is not the same kind of mistake as letting one spam message through.

What Counts as "Good"?

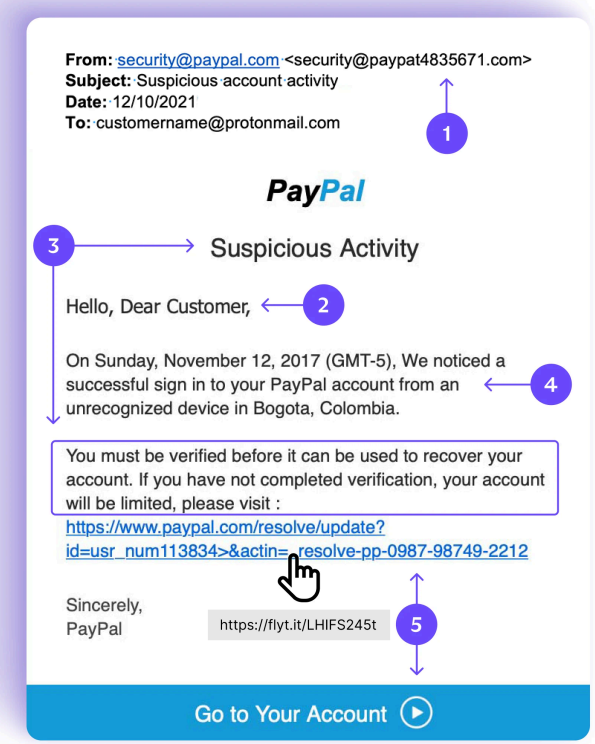
Suppose your spam classifier reaches **97% accuracy**.

Is that good?

Impossible to say, until you know:

- what a trivial model would achieve on the same data
- what the best published system achieves
- what accuracy the application actually requires

A metric is only interpretable against a baseline.



Trivial Baselines

Always compute at least one of these before reporting anything:

Task	Baseline	What it does
Classification	**Majority class**	always predict the most frequent label
Classification	**Random**	predict uniformly at random
Classification	**Stratified random**	predict at random, following the class priors
Regression	**Mean** (or median)	always predict the mean of the training targets

If your model does not beat these, it has learned nothing.

`sklearn.dummy.DummyClassifier` and `DummyRegressor` implement them.

Strong Baselines

Trivial baselines are a sanity check, not a target. In text analytics the *real* baseline is usually:

TF-IDF + Logistic Regression (or **TF-IDF + Ridge Regression** for numeric targets)

- cheap to train, easy to reproduce
- surprisingly hard to beat on many tasks

Rule of thumb: a complex model is only interesting if it beats a *tuned* simple model, not an untuned one.

Evaluation Across Learning Paradigms

What we can measure depends on what supervision is available:

- **Supervised**: the true labels or values are known for the test data, so we compare predictions against them.
  - regression: MSE, MAE, R^2
  - classification: accuracy, precision, recall, F1, ROC-AUC
- **Unsupervised**: no ground truth. We measure properties of the result itself, and read it.
  - clustering: inertia, silhouette
  - topic models: perplexity, coherence

Regression and Unsupervised Metrics

A short tour

Regression: MSE and MAE

Predicting a review's star rating, or a text's readability grade, is a regression problem. On N test documents with true values y_i and predictions $\hat{y}_i$:

$$\mathrm{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2 \qquad \mathrm{RMSE} = \sqrt{\mathrm{MSE}} \qquad \mathrm{MAE} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|$$

- **MSE** is the training loss of linear regression. Squaring makes large errors weigh more, so a few bad misses dominate.
- **RMSE** is in the same units as y, so it is easier to read.
- **MAE** is the most readable: an MAE of 0.6 on star ratings means "off by 0.6 stars on average". It is more robust to outliers.

All three depend on the **scale** of y: an MAE of 0.6 is good on a 1 to 5 scale and poor on a 0 to 1 scale.

Regression: The R^2 Score

To get a measure that does not depend on the scale, compare the model's error with the error of the **mean baseline**:

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} = 1 - \frac{\text{SSE}}{\text{SST}}$$

- $R^2 = 1$: perfect predictions
- $R^2 = 0$: no better than always predicting the mean
- $R^2 < 0$: **worse** than predicting the mean, which can happen on a test set

R^2 is a baseline comparison built into the metric.

On the training set, adding features never decreases R^2, even if they are irrelevant. With thousands of TF-IDF features this makes training R^2 meaningless: report it on **held-out** data.

Clustering: Inertia

Inertia is the k-means loss, computed on the final clusters: the within-cluster sum of squared distances.

$$\text{Inertia} = \sum_{j=1}^{K} \sum_{\mathbf{x} \in S_j} \|\mathbf{x} - \boldsymbol{\mu}_j\|^2$$

- lower means **tighter** clusters
- it always decreases as K grows, so it cannot be minimized to choose K
- instead, plot it against K and look for the **elbow**, where adding clusters stops helping much

It only measures **cohesion**: it says nothing about how well the clusters are separated from each other.

```
KMeans(...).fit(X).inertia_
```

Clustering: Silhouette

The **silhouette** adds separation. For each document d:

- $a(d)$: average distance from d to the other documents of **its own** cluster
- $b(d)$: average distance from d to the documents of the **nearest other** cluster

$$s(d) = \frac{b(d) - a(d)}{\max\{a(d),\, b(d)\}} \in [-1, 1]$$

- $s(d) \approx 1$: well placed; $s(d) \approx 0$: on the border; $s(d) < 0$: probably in the wrong cluster
- averaging over the corpus gives a score, which can be **maximized to choose** K

Both scores are **internal**: they say whether the clusters are compact and separated, not whether they are useful.

- Always read a sample of each cluster.

```
silhouette_score(X, labels, metric="cosine")
```

Topic Models: Perplexity and Coherence

Perplexity on **held-out documents**: how well the model predicts unseen text. Lower is better. It is the same measure as for language models.

Coherence looks at the topics themselves: do a topic's top words actually occur together? One common version averages the PMI of every pair among the topic's top m words, with probabilities estimated from **document co-occurrence**:

$$\text{Coherence}(\text{topic}) = \frac{2}{m(m-1)} \sum_{i<j} \text{PMI}(w_i, w_j)$$

The two can **disagree**: models with lower perplexity can produce topics that people find *less* interpretable. Choose the number of topics on coherence, and check the topics by reading them.

Classification Metrics

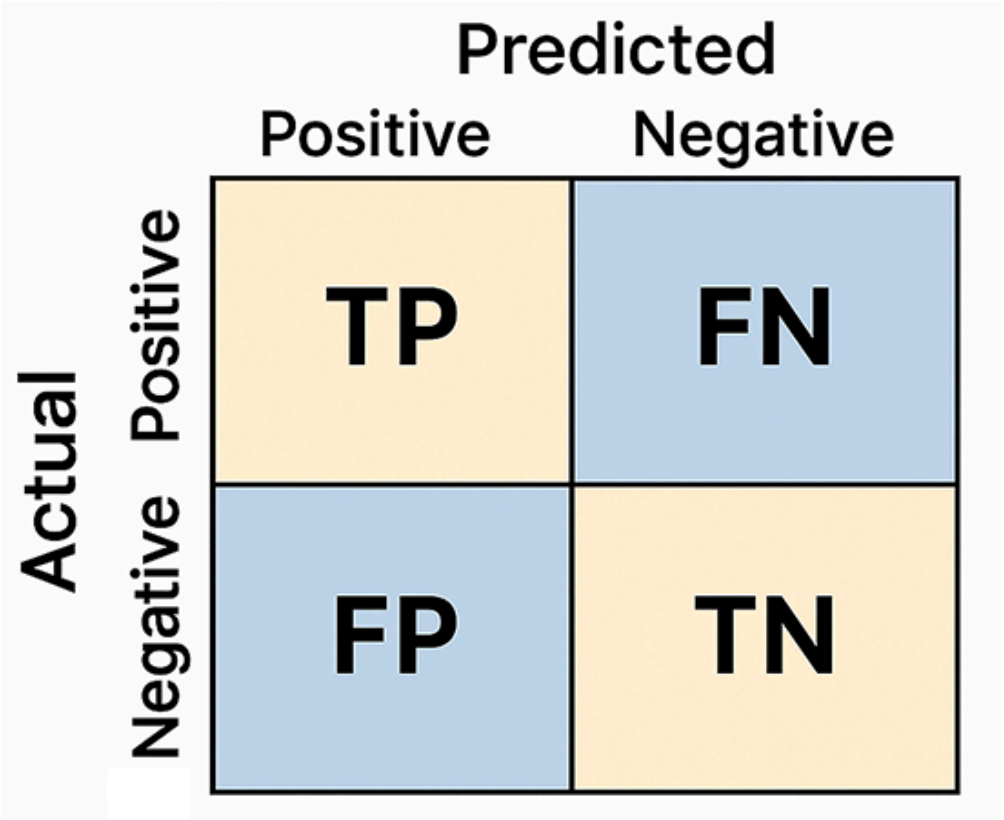
Binary Classification

Confusion Matrix

Evaluation in supervised text classification is based on counting how many predictions are correct and how many are wrong, distinguishing between the two possible classes.

- **TP** = true positives
- **TN** = true negatives
- **FP** = false positives
- **FN** = false negatives

Most classification metrics are defined in terms of these four numbers.



Accuracy

Accuracy measures the fraction of correct predictions:

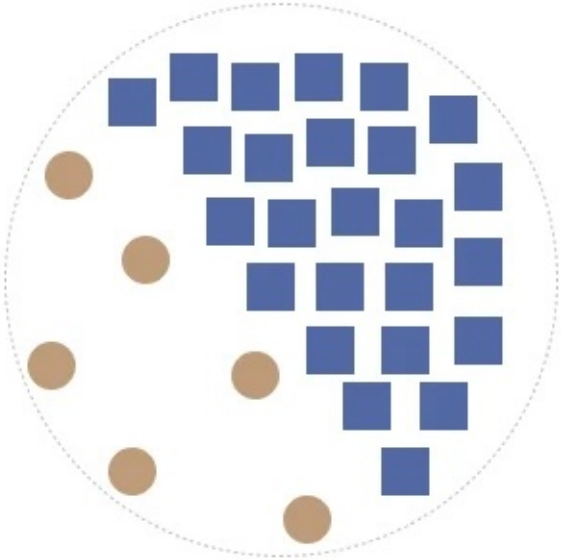
$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

"Out of all predictions, how many did we get right?"

It treats all errors in the same way, which is exactly its weakness.

Imbalanced Data

Text collections are almost always **imbalanced**.



If only 1% of documents belong to the category of interest:

- a model that always answers "no" achieves **99% accuracy**
- and is completely useless

Therefore:

- accuracy alone is misleading whenever classes are skewed
- precision, recall and F1 are usually more informative
- and this is why the majority-class baseline matters

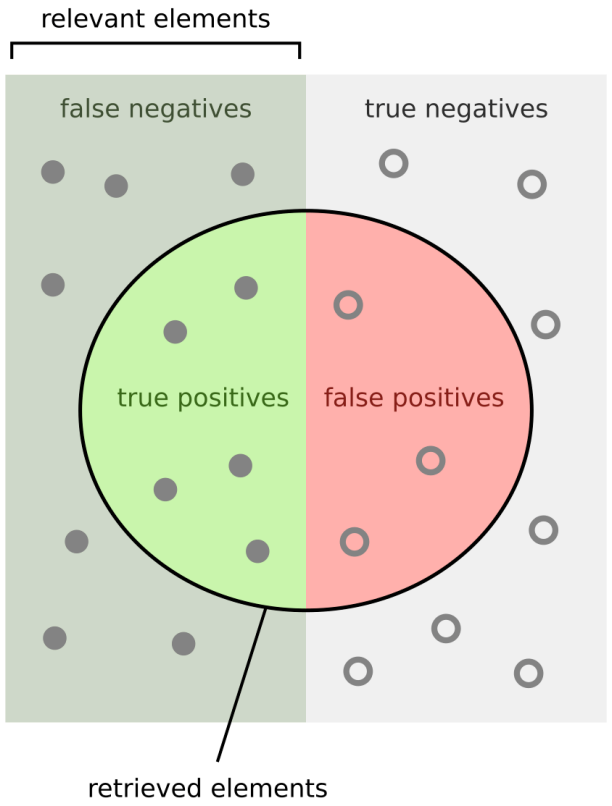
Precision and Recall

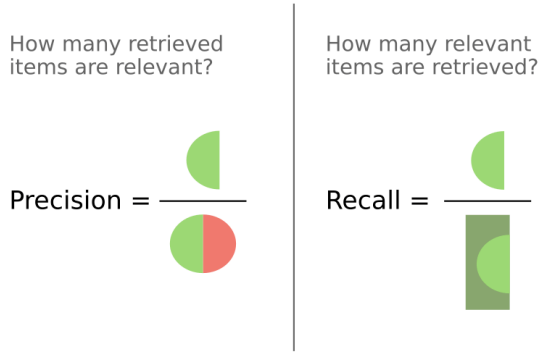
Precision: of everything we labeled positive, how much was right?

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall: of everything that truly was positive, how much did we find?

$$\text{Recall} = \frac{TP}{TP + FN}$$





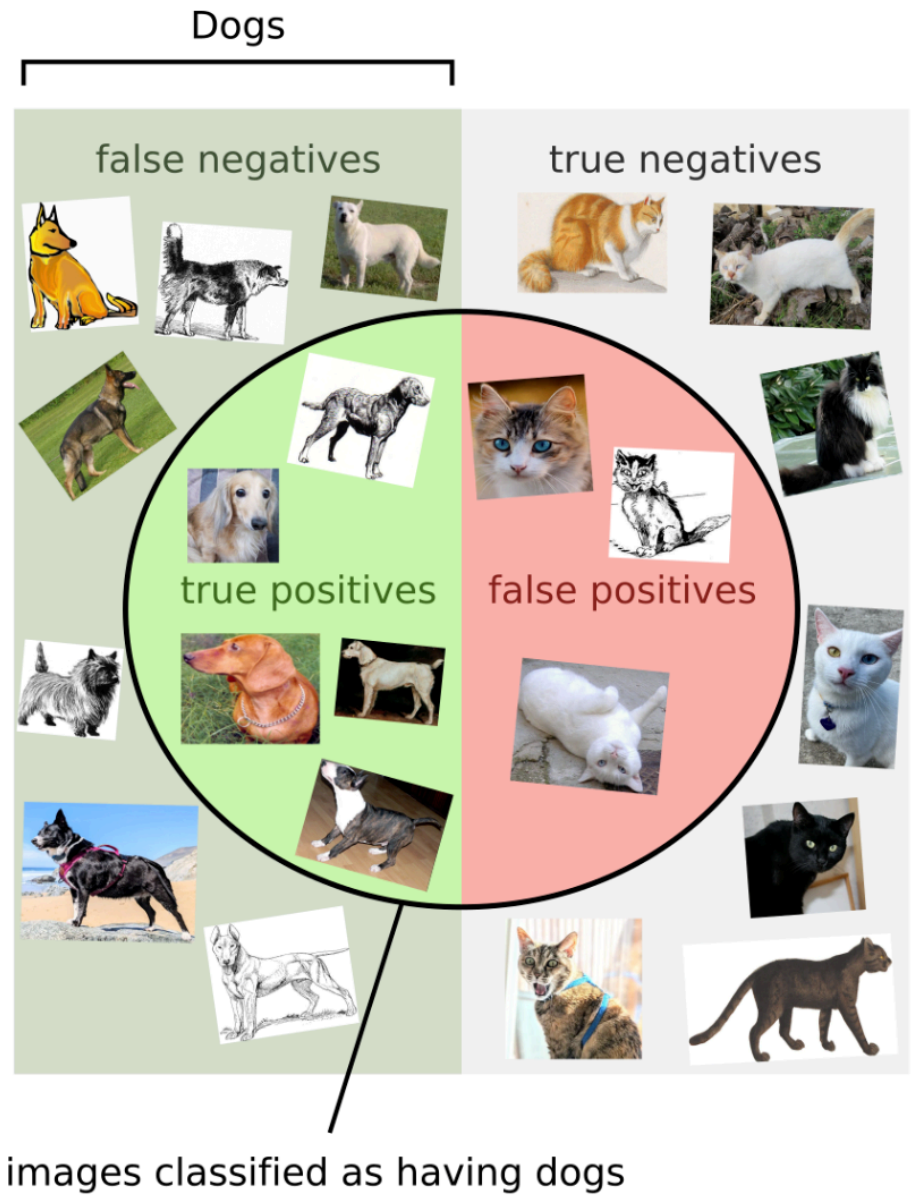
Precision and Recall: Retrieved vs Relevant

These metrics come from information retrieval:

- the set of items **labeled positive** is the **retrieved** set ($TP + FP$)
- the set of items that **truly are positive** is the **relevant** set ($TP + FN$)

$$\text{Precision} = \frac{5\,TP}{5\,TP + 3\,FP}$$

$$\text{Recall} = \frac{5\,TP}{5\,TP + 7\,FN}$$



Precision and Recall in Practice

They pull in opposite directions, and the application decides which one wins:

Application	What hurts more	Optimize
Spam filter	losing a real email (FP)	**precision**
Screening for abusive content	missing a case (FN)	**recall**
Legal / medical document search	missing a case (FN)	**recall**
Ranked search results	irrelevant hits on page 1 (FP)	**precision**

Trivially:

- predict everything positive → recall 1.0.
- predict positive only for the one document the model is most sure about → precision is likely high (1.0 if that one is right), recall close to 0.

F1-score

Because precision and recall can each be gamed, we combine them. The F1-score is their **harmonic** mean:

$$\mathrm{F1} = 2 \cdot \frac{\mathrm{Precision} \cdot \mathrm{Recall}}{\mathrm{Precision} + \mathrm{Recall}}$$

The harmonic mean, unlike the arithmetic mean, is dragged down by the smaller of the two:

- P = 1.0, R = 0.0 → arithmetic mean 0.50, **F1 = 0.0**

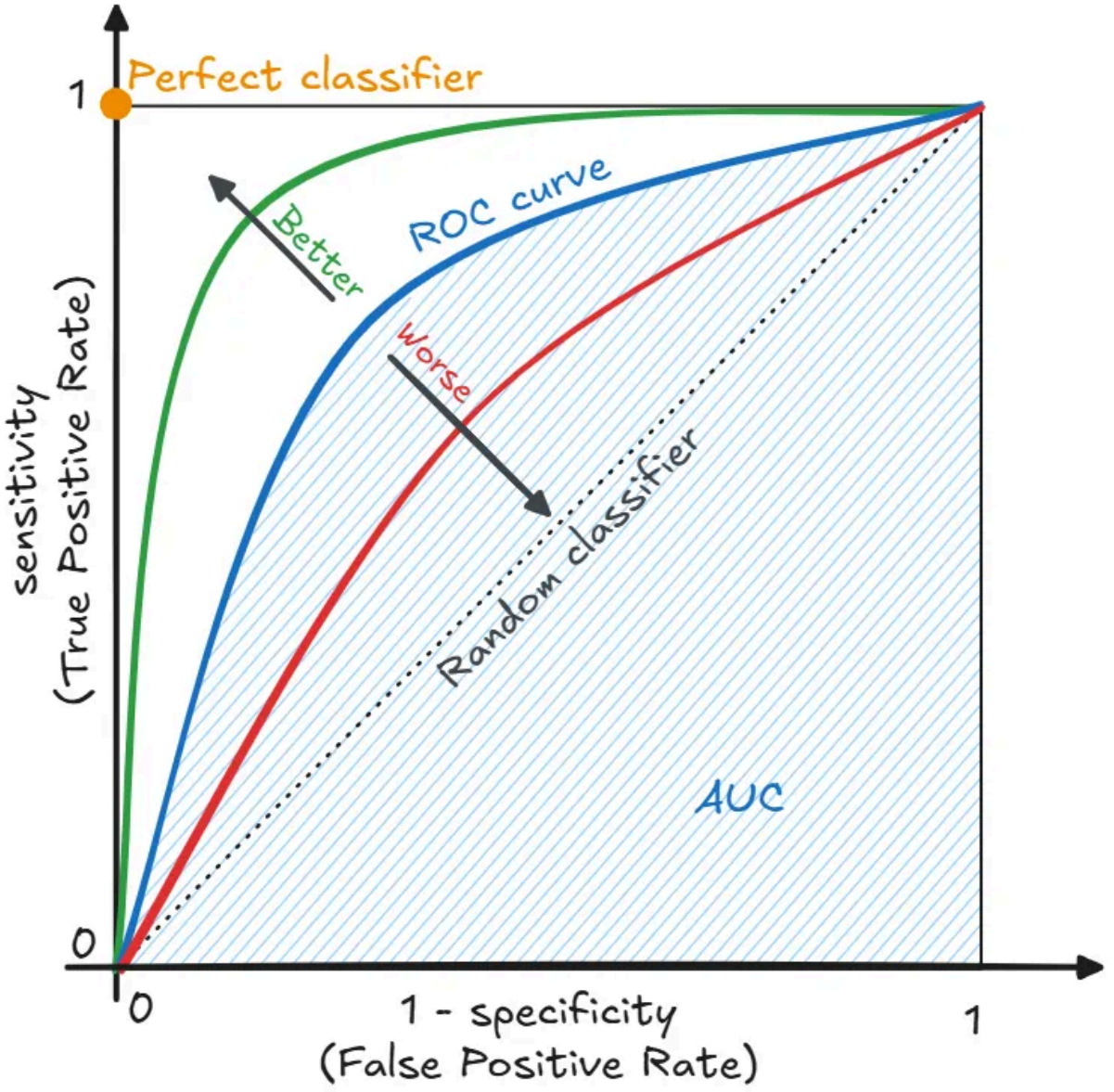
ROC-AUC

All the metrics so far need a **decision threshold**. ROC removes that choice by sweeping it.

The ROC curve plots True Positive Rate against False Positive Rate:

$$\mathrm{TPR} = \frac{TP}{TP + FN}, \quad \mathrm{FPR} = \frac{FP}{FP + TN}$$

AUC is the area under this curve, and measures **ranking quality**.



A Warning About ROC-AUC on Text

ROC-AUC is threshold-free and intuitive, but it is computed over TN as well, and on a heavily imbalanced collection the negative class dominates.

- 1% positives, 99% negatives
- a large absolute number of false positives is still a *small* FPR
- so AUC can look excellent while precision is terrible

For rare-category text problems, prefer the area under the **Precision-Recall curve** (average precision, AP), which ignores TN entirely.

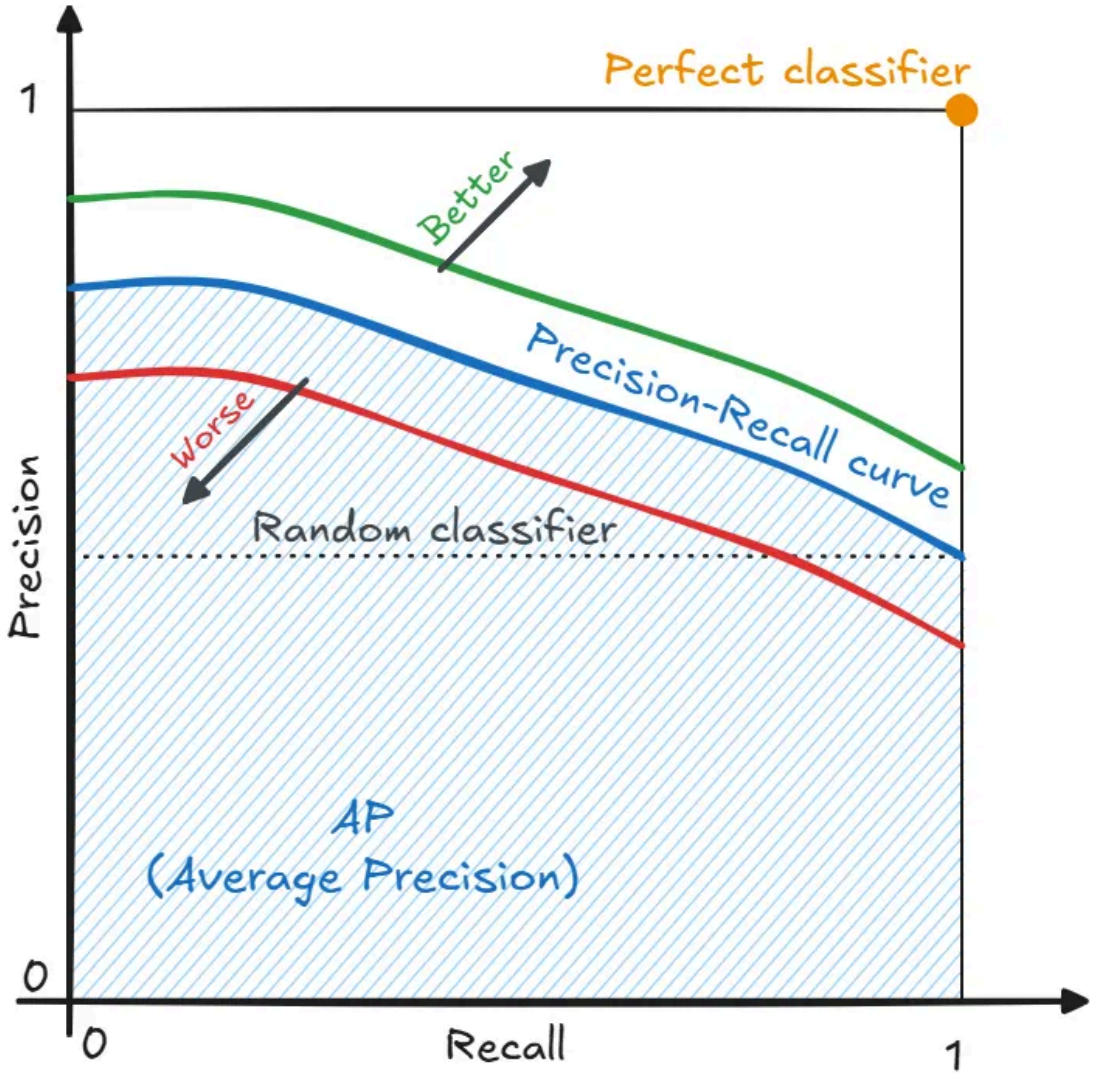
Precision-Recall Curve

It sweeps the decision threshold, but it plots **precision against recall** and never looks at TN.

- lower the threshold: **recall goes up**, precision usually goes down
- raise the threshold: precision usually goes up, recall goes down

The area under the curve is the **average precision** (AP).

- the baseline is not 0.5 but the **positive rate** (a horizontal line at $\frac{TP+FN}{N}$)
- the perfect classifier reaches $(1, 1)$



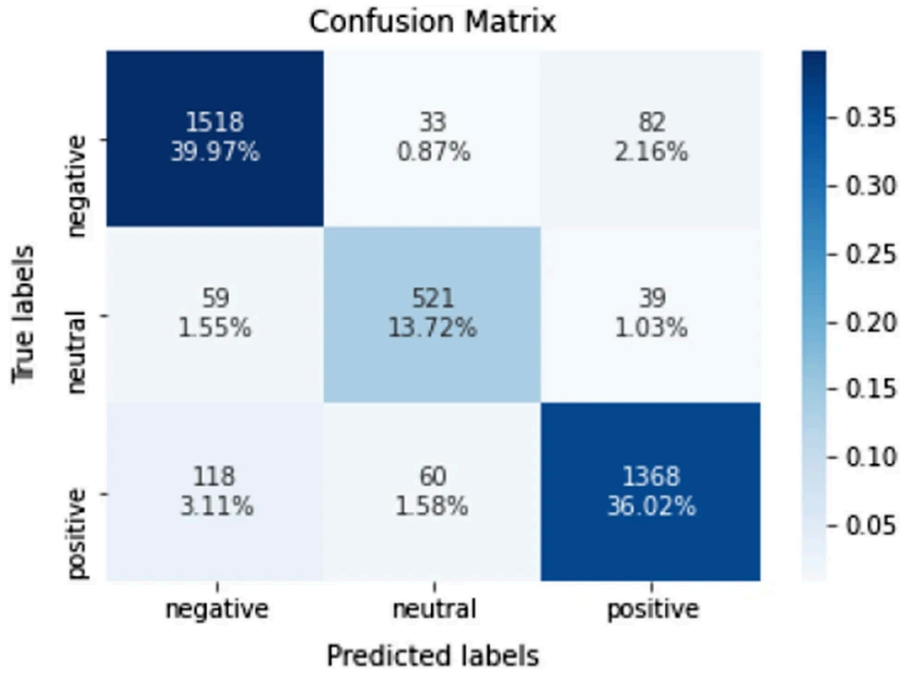
Classification Metrics

Beyond Binary

Multiclass Confusion Matrix

In a multiclass problem, the confusion matrix summarizes how often documents of true class c are assigned to predicted class c'.

- each cell (c, c') counts documents with true label c and predicted label c'
- the diagonal is correct predictions; everything off-diagonal is an error
- **raw counts** show absolute frequencies, dominated by large classes
- **row-normalized** values give each true class equal importance



It is the most informative single object in evaluation: it shows *which* classes get confused with *which*, not just how often.

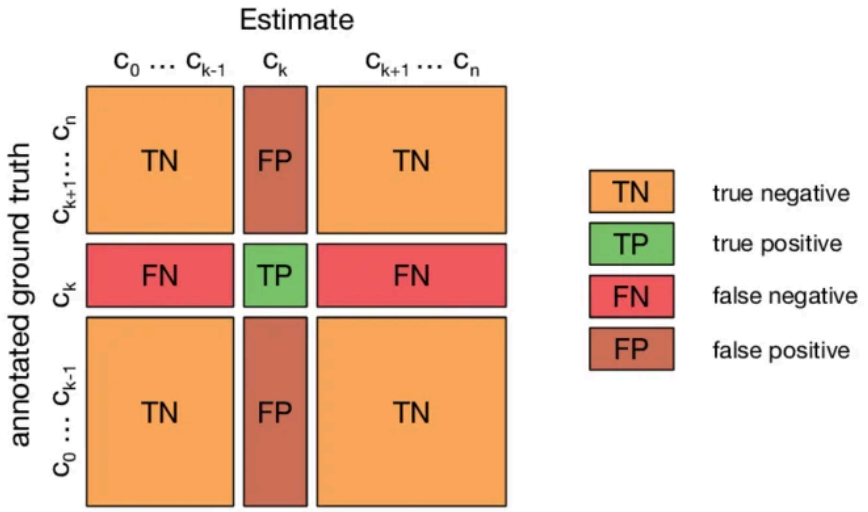
Micro- and Macro-Averaging

Precision, recall and F1 are defined for a *binary* problem. With $|Y|$ classes, each class $c \in Y$ is treated as its own binary problem, with its own contingency table: TP_c, FP_c, FN_c, TN_c.

To get one global number, we aggregate. Two strategies:

- **micro-averaging**: pool the counts, then compute the metric
- **macro-averaging**: compute the metric per class, then average

They answer different questions and can disagree sharply.



Micro-Averaging

The per-class contingency tables are summed into a single global table, and the metric is computed on that:

$$\text{Precision}_{\text{micro}} = \frac{\sum_{c \in Y} TP_c}{\sum_{c \in Y} (TP_c + FP_c)} \qquad \text{Recall}_{\text{micro}} = \frac{\sum_{c \in Y} TP_c}{\sum_{c \in Y} (TP_c + FN_c)}$$

$$\text{F1}_{\text{micro}} = \frac{2 \cdot \text{Precision}_{\text{micro}} \cdot \text{Recall}_{\text{micro}}}{\text{Precision}_{\text{micro}} + \text{Recall}_{\text{micro}}}$$

Every **instance** contributes equally, so frequent classes dominate.

In a single-label multiclass problem every FP for one class is an FN for another, so micro-precision, micro-recall and micro-F1 all equal **accuracy**.

Macro-Averaging

Metrics are computed separately for each class, then averaged:

$$\text{Precision}_{\text{macro}} = \frac{1}{|Y|} \sum_{c \in Y} \frac{TP_c}{TP_c + FP_c} \qquad \text{Recall}_{\text{macro}} = \frac{1}{|Y|} \sum_{c \in Y} \frac{TP_c}{TP_c + FN_c}$$

$$\text{F1}_{\text{macro}} = \frac{1}{|Y|} \sum_{c \in Y} \frac{2 \cdot \text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}$$

Every **class** contributes equally, regardless of size. A rare class that the model ignores completely will drag the score down.

When to Use Which

Macro-averaging treats each class equally. Use when:

- all categories are equally important
- the dataset is imbalanced and minority classes must count fairly

Micro-averaging treats each instance equally. Use when:

- you care about the total number of misclassifications
- you want a score that improves whenever total errors drop

In short: **macro is fair across classes, micro is fair across instances.** On imbalanced text data, report macro-F1; if you only report one number, that is usually the honest one.

Weighted Averaging

A third option sits between them: average the per-class metrics, weighted by class support (n_c of the N test documents belong to class c).

$$\mathrm{F1}_{\text{weighted}} = \sum_{c \in Y} \frac{n_c}{N} \cdot \mathrm{F1}_c$$

- closer to micro in behavior on imbalanced data

Reporting all three costs nothing and immediately reveals whether your model is quietly failing on the small classes.

Experimental Protocols

If it sounds too good to be true...

Experimental Protocols

Experiments in a controlled environment let us **predict the effectiveness of a method** and compare competing ideas.

In text analytics, that controlled environment is one or more **corpora** designed to model the task of interest.

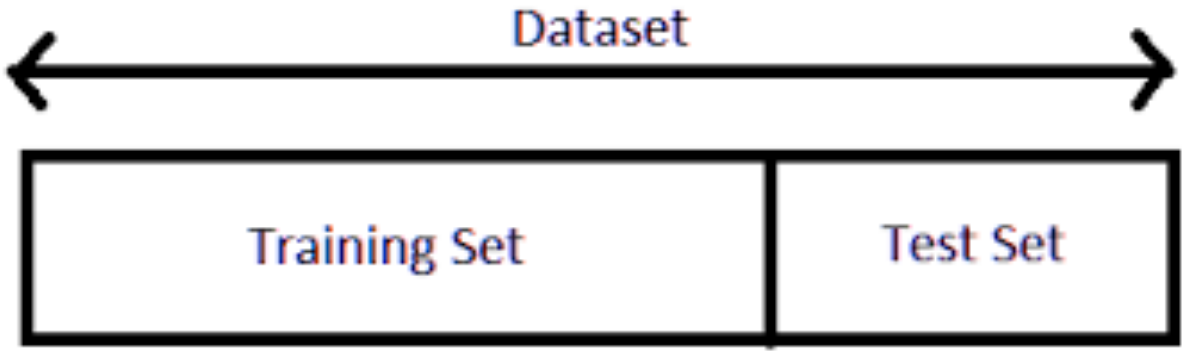
The same corpus can be exploited in many ways, following different protocols. Every protocol trades off:

- computational cost
- reproducibility
- statistical relevance of the results

Train and Test

The simplest evaluation setup splits the data into:

- a **training set**, used to fit the model
- a **test set**, used to estimate generalization performance

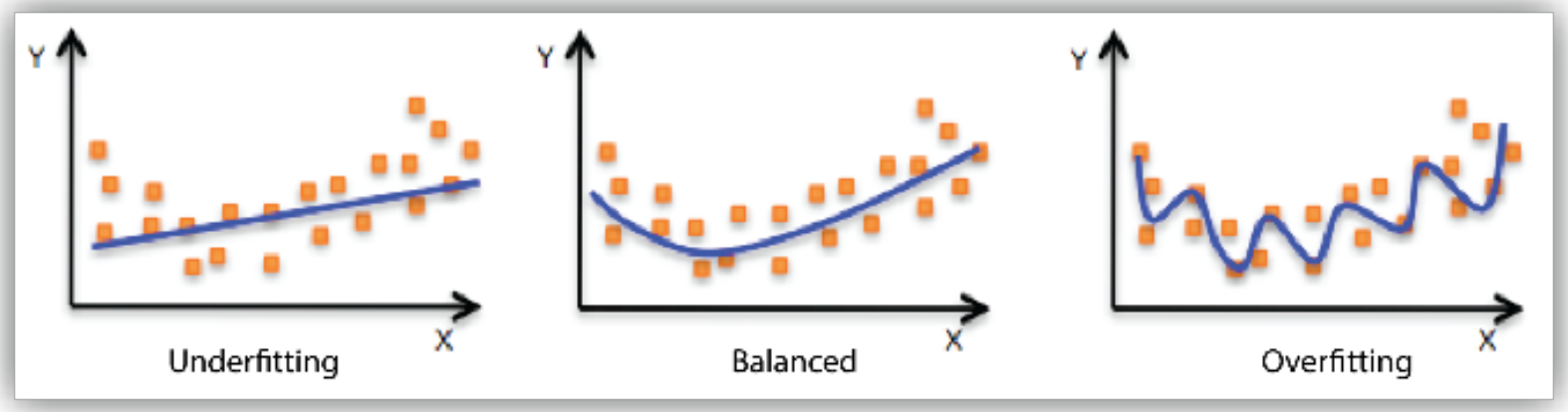


Information from the test set must **never** be used to train the model, or in any decision about the training process; otherwise the test results stop being representative of performance on new data.

Typical Problems

When training models, two main problems can occur:

- **underfitting**: the model is too simple
- **overfitting**: the model is too complex



Underfitting and Overfitting

Underfitting

The model cannot capture the structure of the data.

- high training error
- high test error

Usually **the model is too simple**, or not trained enough.

Overfitting

The model fits the training data very well but performs poorly on new documents.

- very low training error
- much higher test error

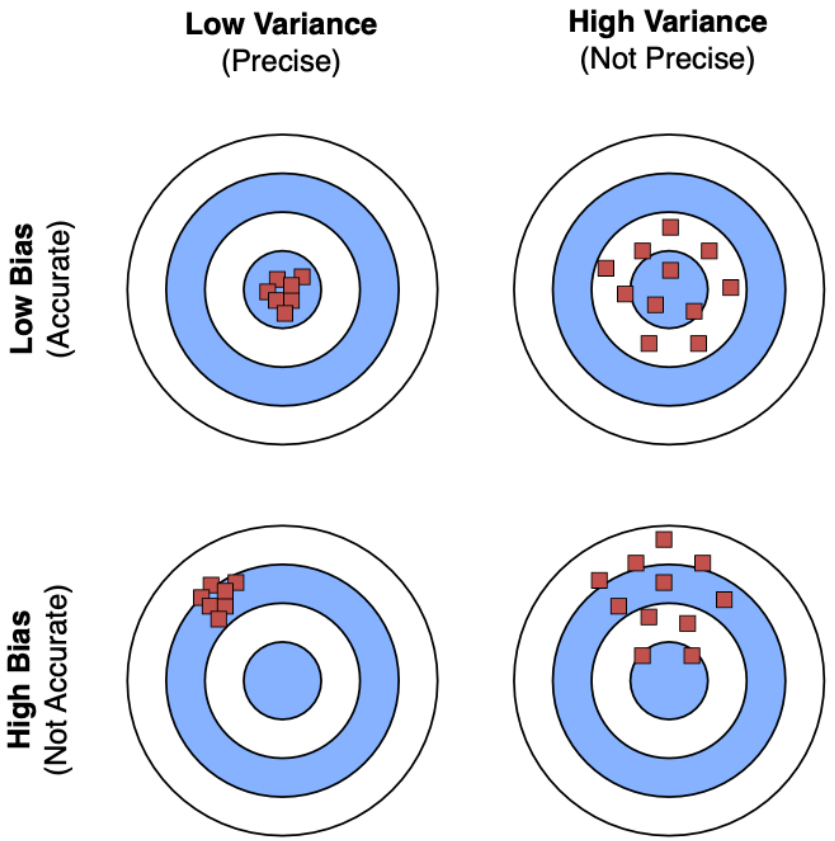
The model has learned noise and corpus-specific details instead of general patterns.

With bag-of-words overfitting is easy: vocabularies are large, so a model can memorize rare words that correlate with a label in the training corpus only.

The Bias-Variance Trade-off

Under and overfitting have a more general explanation. Imagine retraining the model many times, each time on a different sample of documents from the same source, and looking at its prediction for the same test document:

- **High bias** (→ underfitting): the prediction for that document is **systematically wrong**, whatever the training set, because the model is too simple to capture the pattern.
- **High variance** (→ overfitting): the prediction for that document **changes a lot**, because the model is too sensitive to the particular documents it was trained on.



Text sits on the high-variance side: tens of thousands of words and only few documents contain each word.

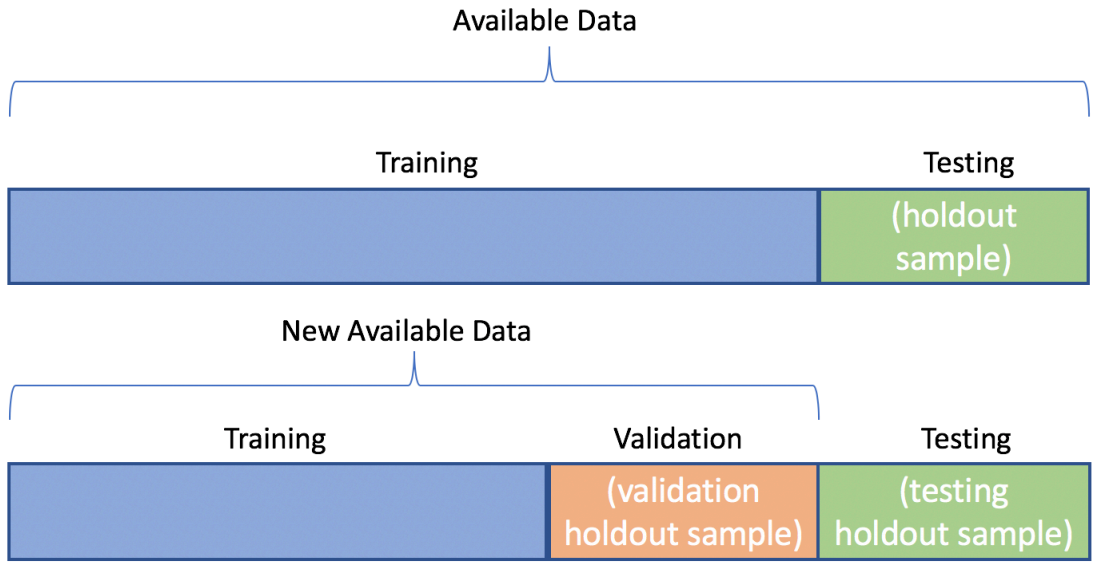
Hold-out Validation

If we only use a training set and a test set, we are tempted to: try many models, try many hyperparameters, and keep whichever works best on the test set: this makes the test set **no longer a fair estimate of generalization**.

To avoid this problem, the simplest strategy is to split the data into train, validation and test sets.

The model is:

- trained on the training set,
- tuned on the validation set,
- and evaluated on the test set.



Train, Validation, Test

Training set

Used to fit the model **parameters** (learned by the model). Typically 70-80% of the data.

Validation set

Used to select models and **hyperparameters** (selected by the user). Never seen during training.

Test set

Used **only once**, to estimate the final generalization performance.

Hold-out: Pros and Cons

Data is split once and for all. Standard in text analytics: Reuters-21578, RCV1-v2, IMDB.

Pros

- easy to reproduce: everyone uses the same split
- the right choice for time-ordered data (training data precedes test data)
- experiments are quick to run

Cons

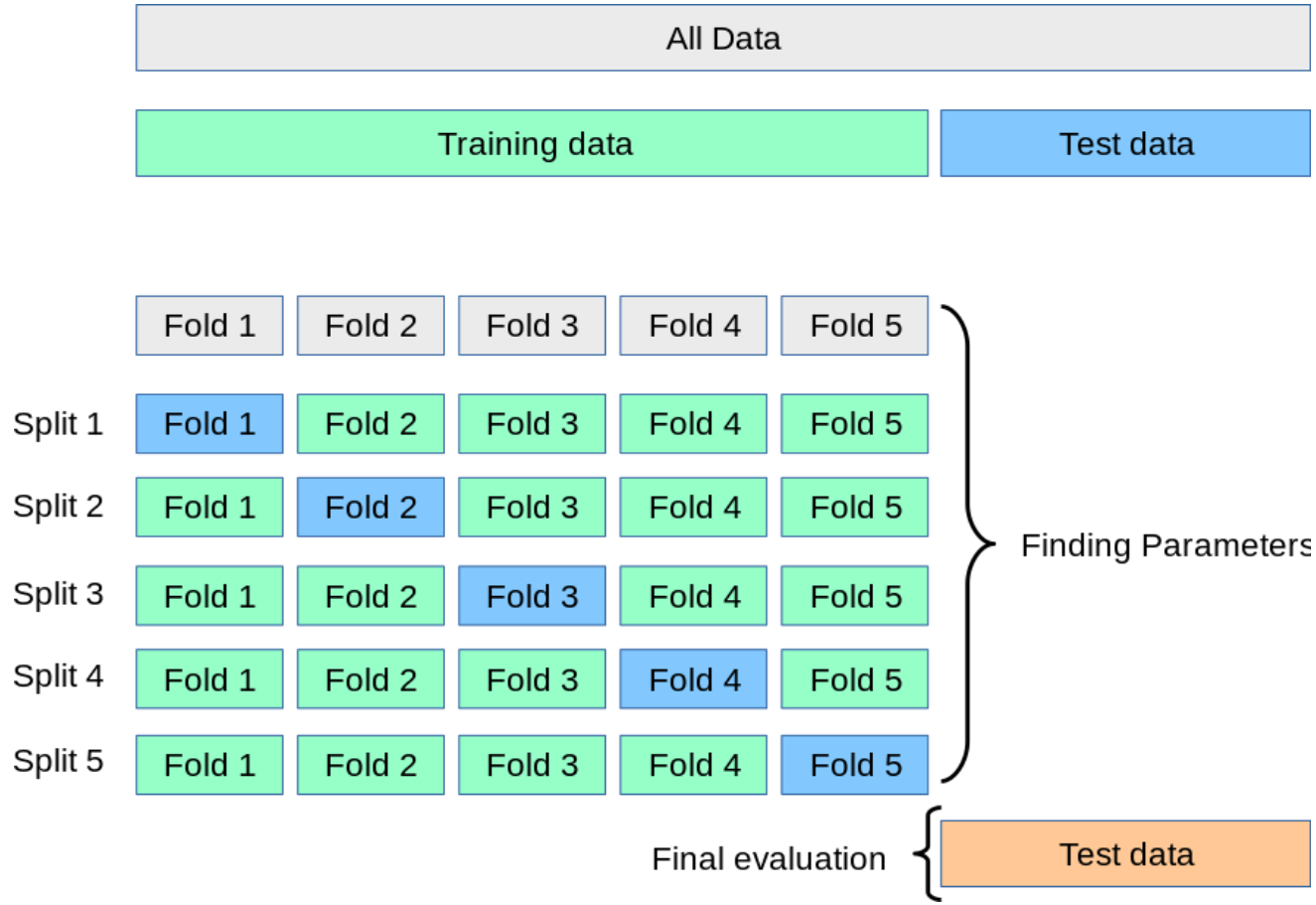
- risk of overfitting the validation data over many iterations
- risk of low statistical relevance, unless the test set is large

K-Fold Cross-Validation

The training data is split into K folds. For each fold:

- train on $K - 1$ folds
- evaluate on the remaining one

The final score is the average over the K runs, which gives a more stable estimate than a single hold-out split.



- **Pros:** every training document is used for validation exactly once.
- **Cons:** cost grows linearly with the number of folds; splits must be reported to be reproducible.

Stratification

Many text classification problems are **highly unbalanced**: very few positive examples for a label.

- A plain K-fold split then risks producing folds where nearly all examples of a class land in one part, making that split useless for evaluation.
- In the worst case a fold contains *zero* positives, and recall is undefined.

Stratified protocols keep the class ratio constant across splits. For text, this should be the default, not an option.

Other Protocols

Leave-one-out

K-fold taken to the extreme: $K = N$, so each test set is a single document.

- *Pros:* trivially reproducible, good statistical relevance.
- *Cons:* very high cost, impractical for most corpora.

Random sampling

Fix a split proportion (e.g. 80/20) and repeat the random split an arbitrary number of times, recording the scores.

- *Pros:* high statistical relevance, cost is flexible.
- *Cons:* hard to reproduce exactly, needs statistical analysis to summarize.

A Text-Specific Warning: Grouped Splits

Random splitting assumes documents are independent. In text corpora they often are not:

- multiple reviews by the **same author**
- multiple articles from the **same source** or **same event**
- near-duplicate documents (retweets, syndicated news, boilerplate)

If related documents land on both sides of the split, the model can recognize the *source* rather than solve the *task*.

You can perform K-Fold by splitting by author, source, or thread, instead of document.

`GroupKFold` or `StratifiedGroupKFold` in sklearn

What Has to Be Chosen

We can use evaluation protocols to select models and hyperparameters in a text analytics pipeline such that the performance on a given metric is maximized. Common choices are:

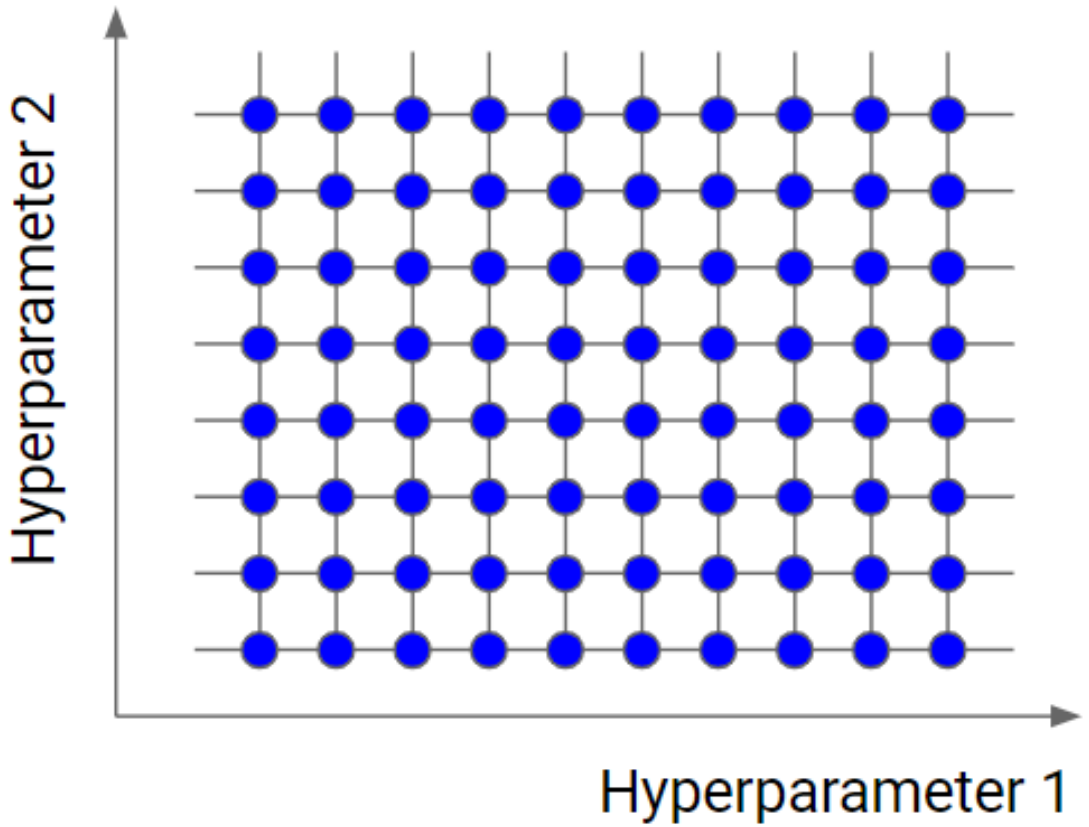
- which features to extract? words, n-grams, characters?
- lowercasing, stopword removal, stemming or lemmatization?
- TF-IDF weighting scheme, minimum document frequency
- use of POS tagging or parsing, and how?
- feature selection: which measure, and how many features?
- the learner and its own hyperparameters

Which is the optimal choice for a specific task?

Grid Search

Grid search is a systematic hyperparameter optimization technique used to identify the best combination of parameters.

- define a discrete set of candidate values for each hyperparameter
- construct a **grid of all possible combinations**
- train and evaluate the model for each combination
- select the combination with the best **validation** performance



Grid Search Explodes

Optimization is always done against a specific evaluation measure. But the number of combinations grows multiplicatively. For example:

- **5 feature types**, tested individually, all together, and in all pairs: $5 + 1 + 10 = 16$
- **5 feature selection levels** (all features, top-k, ...)
- **10 values** for the regularization strength C of a logistic regression

$$16 \times 5 \times 10 = \mathbf{800}\ \text{configurations}$$

And each of these must be trained and evaluated, multiplied again by K if you cross-validate.

C is scikit-learn's name for the inverse regularization strength, $C = 1/\lambda$: it is not the corpus.

Tips on Optimization

- **Optimize in sequence.** Parameters with loose correlation can be tuned one group at a time, e.g. first the feature selection amount, then C. Fewer combinations, faster search.
- **Coarse to fine.** For parameters with many possible values, e.g. the number of neighbors k of a k-nearest-neighbors classifier, search in two steps:
  1. coarse: $k \in \{1, 5, 10, 15, 20, 25, 30, 35, 40\}$
  2. refinement: $k \in \{6, 7, 8, 9, 11, 12, 13, 14\}$
- **Use a logarithmic scale** for numeric parameters spanning orders of magnitude:

$$C \in \{0.001,\ 0.01,\ 0.1,\ 1,\ 10,\ 100,\ 1000\}$$

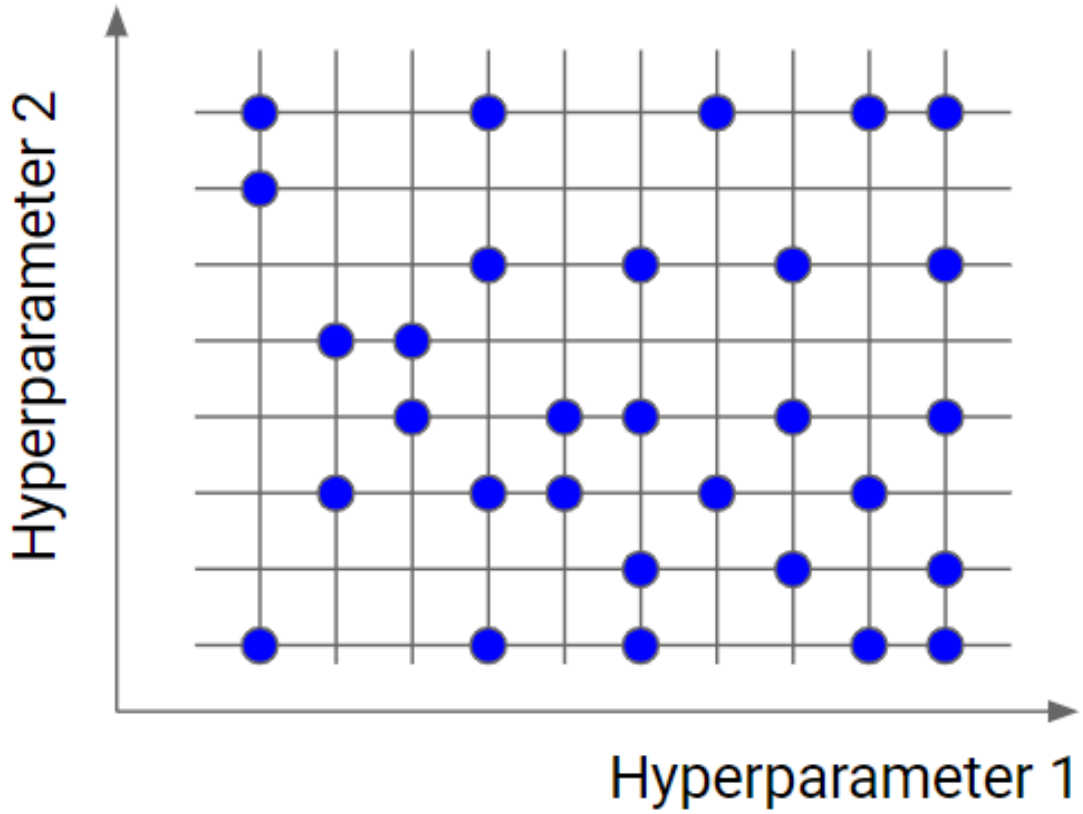
Random Search

Instead of testing every hyperparameter combination:

- we can **sample** them,
- drawing configurations at random from the grid
- until a fixed experiment budget is spent.

Random search is usually the better default

- when only a few hyperparameters actually matter,
- sampling explores more distinct values of *those* for the same compute.



How Should the Test Set Be Used?

Cross-validation is used for:

- model selection
- hyperparameter tuning

Then you retrain on the full training dataset and use the test set **only once, at the very end.**



The test set is

- **not** used in training,
- **not** used in cross-validation

Otherwise, the test performance is not an unbiased generalization estimate.

The same holds without labels: e.g., choose the number of topics on validation documents.

Data Leakage

Data leakage happens when information from the test set is used during training.

Examples:

- normalizing or fitting a vectorizer using the full dataset before splitting
- feature selection using all samples
- using test data to choose hyperparameters

Consequence: the evaluation becomes overly optimistic and does not reflect real-world performance. The model looks excellent in the notebook and fails in deployment.

Data Leakage Quiz

Which of these is wrong, and why?

```
# option A
vectorizer = TfidfVectorizer()
X_all = vectorizer.fit_transform(documents)
X_train, X_test, y_train, y_test = train_test_split(X_all, y)
clf = LogisticRegression().fit(X_train, y_train)
```

```
# option B
X_train, X_test, y_train, y_test = train_test_split(documents, y)
vectorizer = TfidfVectorizer()
X_train = vectorizer.fit_transform(X_train)
X_test = vectorizer.transform(X_test)
clf = LogisticRegression().fit(X_train, y_train)
```

Leakage Specific to Text

Beyond the vectorizer, text corpora leak in ways tabular data does not:

Source	What happens
Duplicate documents	the same text appears in train and test: the model retrieves rather than generalizes
Temporal leakage	training on later documents than the test ones; the model exploits knowledge of future events
Author / source overlap	the model learns the writing style of a source, not the task
Label in the text	e.g. a review that literally contains its own star rating
Pretrained models	the test corpus may already be in the pretraining data of a language model

That last row will come back when we discuss LLM benchmarks.

A Final Checklist

Before reporting a result, check that:

1. a **baseline** is reported alongside the model
2. the **metric** matches the task and the class distribution (macro-F1 for imbalanced text)
3. the split is **stratified**, and **grouped** if documents share authors or sources
4. every fitted transformation lives **inside** the cross-validation loop
5. the **test set** was used exactly once
6. the result is reported with some measure of **variability** across folds or seeds

Sources: Textbooks

- Jurafsky & Martin, *Speech and Language Processing*, 3rd ed. (Jan 2026 draft), ch. 3.2-3.3, 3.5 (perplexity, generalizing vs overfitting), ch. 4.5 (cross-entropy loss), 4.9-4.10 (precision, recall, F-measure; test sets and cross-validation)
- Manning, Raghavan & Schütze (2008), *Introduction to Information Retrieval*, ch. 8.3 (unranked evaluation), 13.6 (evaluation of text classification, micro- and macro-averaging)
- James, Witten, Hastie, Tibshirani & Taylor, *An Introduction to Statistical Learning with Applications in Python*, ch. 2.2 (assessing model accuracy, bias-variance trade-off), 3.1.3 (R^2)